



សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

Royal University of Phnom Penh

Design Documentation

1. Table of Contents

1.	<i>Table of Contents</i>	<i>i</i>
1.	<i>Introduction.....</i>	<i>1</i>
1.1.	Purpose	1
1.2.	Overview of the Architecture	1
1.3.	Presentation Layer (Frontend).....	2
1.4.	Application Layer (Backend).....	3
1.5.	Data Layer (Database)	4
1.6.	Interaction Between Layers	5
1.7.	Benefits of the Architecture	6
2.	<i>Data Model / Database Design</i>	<i>7</i>
2.1.	Entity Relationship (ER) Diagram.....	7
2.2.	Entity: User	8
2.3.	Entity: Wallet	9
2.4.	Entity: Transaction.....	9
2.5.	Entity: Ads	10
2.6.	Entity: adsTransaction	11
2.7.	Entity: LedSchema	12
3.	<i>ER diagrams, schema, and relationships.....</i>	<i>13</i>
3.1.	Relationship Summary.....	14
4.	<i>UML Diagrams.....</i>	<i>15</i>
4.1.	Class diagrams	15
4.2.	User Class	15
4.3.	Wallet Class	16
4.4.	Transaction Class.....	17
4.5.	Ads Class.....	17
4.6.	LedSchema Class	18
4.7.	AdsTransaction Class	19

4.8. Class Diagram Structure	20
5. <i>API Design Document</i>	21

1. Introduction

1.1. Purpose

Section នេះពន្យល់ពីរបៀបដែល Advertisement System Website ត្រូវបានរៀបចំឡើង ហើយ Advertisement System ត្រូវបានបង្កើតឡើងដោយប្រើប្រាស់ Three-Tier Architecture ដែលបែងចែក Application ជាបី Layers សំខាន់ៗមានដូចជា Presentation Layer, Application Layer, និង Data Layer ដើម្បីអោយ System នោះមានសមត្ថភាព និង អាចប្រើប្រាស់បាន។ ដែល Architecture មានដូចជា (Frontend), Application Logic (Backend), និង Data Management (Database) ត្រូវបានរៀបចំ និងអនុញ្ញាតឱ្យ Layers នីមួយៗអាច Developed, Updated, និង Maintained ដោយមិនប៉ះពាល់ដល់ System ទាំងមូលទេ។ តាមរយៈ Structure នេះ System កាន់តែមានប្រសិទ្ធភាព ងាយស្រួលដោះស្រាយបញ្ហា ដែល Layers នីមួយៗ ដំណើរការនិងទាក់ទងគ្នាបានល្អ និងមិនប៉ះពាល់ដល់ Layers ណាមួយឡើយ។

1.2. Overview of the Architecture

System នេះ follows three-tier architecture model៖

- Presentation Layer (Frontend): User Interface (UI) ជាកន្លែងដែល User អាច Interact ជាមួយ។
- Data Layer (Database): Data Storage Layer ទិន្នន័យទាំងអស់ត្រូវបាន រក្សាទុកដោយសុវត្ថិភាព។
- Application Layer (Backend): Core Logic អាច Handles System Operations, Authentication, និង Data Processing.

Layers នីមួយៗប្រាស្រ័យទាក់ទងជាមួយគ្នា និង តាមរយៈ Interface ដែលបានកំណត់ ហើយអនុញ្ញាតឱ្យងាយស្រួល Modification ឬ Replacement ដោយមិនប៉ះពាល់ដល់ Layers ផ្សេងទៀតឡើយ។

1.3. Presentation Layer (Frontend)

Technology Used: Next.js, Tailwind CSS, HTML, JavaScript.

ផ្នែក Frontend ផ្តល់នូវ Interface ដល់ User យ៉ាងសំខាន់សម្រាប់អ្នកផ្សាយពាណិជ្ជកម្ម និងអ្នកប្រើប្រាស់ដើម្បី Interact ជាមួយ System ហើយអនុញ្ញាតឱ្យអ្នកប្រើដូចជា៖

- Register និង Login Accounts របស់ User។
- Upload និង Manage Advertisement Content របស់ User ដូចជា Images, Videos, or Text។
- View Ad Slides Dynamically Displayed នៅលើ Website (Simulating IoT display)។

Design Notes៖

- Build ជាមួយ Next.js សម្រាប់ Fast, Server-Rendered Pages។
- ប្រើ Tailwind CSS សម្រាប់ Design និង Responsive Styling។
- Includes Interactive Slides លើ Homepage ដើម្បីបង្ហាញការផ្សាយពាណិជ្ជកម្ម (IoT display concept)។

1.4. Application Layer (Backend)

Technology Used: Node.js, Express.js.

ផ្នែក Backend ទទួលខុសត្រូវលើ Handling Business Logic និង Data Flow រវាងផ្នែក Frontend និង Database។ វា Process លើ Requests, Validates Input, Manages Sessions និង Communicates ជាមួយ MongoDB Database។

Main Functionalities:

- User Management: Registration, Login និង Authentication ដោយប្រើ

Secure Password Hashing។

- Advertisement Management: Upload, Update និង Delete Advertisements។
- API Handling: Provides RESTful APIs for Frontend Requests។
- Data Validation & Security: ប្រើ Express Middleware សម្រាប់ Request

Validation និង Secure Handling of Inputs។

1.5. Data Layer (Database)

Technology Used: MongoDB (NoSQL Database).

ផ្នែក Data Layer ទទួលខុសត្រូវលើការ Store Data, Retrieve Data, និង Manage Data ដែលទាក់ទងនឹង System ដូចជា User, Advertisement, និង Transaction។ វាជាប្រភពទិន្នន័យសំខាន់សម្រាប់ Backend ដើម្បី Request ពី Frontend។

Main Functionalities

- User Collection: ផ្ទុកព័ត៌មានអ្នកប្រើប្រាស់ (User Profiles) រួមទាំង Username, Email, Password (Encrypted), និង Role។
- Advertisement Collection: ផ្ទុកទិន្នន័យនៃការផ្សាយពាណិជ្ជកម្ម រួមមាន Title, Media URL (Image/Video), Type, Duration, និង Publisher ID។
- LED / Display Collection: ផ្ទុកព័ត៌មានអំពី LED Screens ដូចជា Name, Location, Size, និង Status (Active/Inactive) ដើម្បីសម្រួលការបង្ហាញ Ads។
- Transaction Collection: កំណត់ប្រតិបត្តិការផ្សាយពាណិជ្ជកម្ម (Ad Transactions) រួមមាន Ads ID, Publisher ID និង Timestamp សម្រាប់ Tracking និង Reporting។

Design Notes

- ប្រើ MongoDB ជា NoSQL Database ដែលផ្តល់ភាព Flexible និង Scalable សម្រាប់ IoT និង Web Applications។
- ប្រើ Mongoose ORM ដើម្បីកំណត់ Schema Structure និងធានា Data Consistency។
- ទិន្នន័យត្រូវបានផ្ទុកជាឯកសារ Document-Based JSON ដើម្បីងាយស្រួល Query និង Integration ជាមួយ Node.js APIs។
- មានការអនុវត្ត Indexing សម្រាប់ Query Performance និងការពារ Data Integrity តាមរយៈ Validation Rules នៅក្នុង Schema។

1.6. Interaction Between Layers

- User Interaction (Frontend)៖

អ្នកប្រើប្រាស់ចូលទៅកាន់ website និង Registers an Account ដើម្បីអាច uploads advertisements តាមរយៈ UI។

- Data Processing (Backend)៖

Express Server ទទួលនូវ User's Request, validates និង Data, ហើយនិងផ្ញើវាទៅកាន់ MongoDB សម្រាប់ Store។

- Data Storage and Retrieval (Database)៖

MongoDB រក្សាទុកទិន្នន័យរបស់ User និង Advertisement ។ នៅពេលចាំបាច់ វានឹង Retrieves Data ហើយ Return វាទៅផ្នែក Backend វិញ។

- IoT Simulation (Ad Display)៖

Frontend រួមបញ្ចូលផ្នែក Slideshow Section ដែលបង្ហាញការផ្សាយពាណិជ្ជកម្មដែលបាន Uploaded Automatically នៅលើ IoT Display Board(Simulates Digital Screen)។

1.7. Benefits of the Architecture

- Scalability ៖ Layers នីមួយៗអាចត្រូវបាន Upgraded ឬធ្វើ Scaled Independently។
- Security ៖ ទិន្នន័យត្រូវបាន Validated និង Protected នៅក្នុងផ្នែក Backend មុនពេលទៅដល់ Database។
- Maintainability ៖ Modular Structure អនុញ្ញាតឱ្យ Developers ធ្វើការ Update នូវ Individual Components យ៉ាងងាយស្រួល។
- Performance ៖ Next. Optimizes ផ្នែក Frontend Performance នឹង Fast Rendering ហើយនិង Dynamic Updates។

សរុបមក Architecture Document ត្រូវបាន Designed ឡើងដោយប្រើ Structured Three-Tier Architecture ដែលបែងចែកយ៉ាងច្បាស់នូវ Presentation, Application និង Data Layers។ ដោយការរួមបញ្ចូល Next.js ជាមួយ Tailwind CSS សម្រាប់ផ្នែក Frontend, Node.js និង Express.js សម្រាប់ផ្នែក Backend Logic និង MongoDB សម្រាប់ផ្ទុកទិន្នន័យដែលអាចទុកចិត្តបាន System ធានានូវ Performance, Scalability និង Maintainability។ ការរចនានេះអនុញ្ញាតឱ្យអ្នកប្រើប្រាស់ងាយស្រួល Register Upload និង Manage Advertisement Content។

2. Data Model / Database Design

Section នេះ Describes អំពីរបៀបដែល Data របស់ Advertisement System ត្រូវបានរៀបចំ និងរក្សាទុកនៅក្នុង MongoDB។ វាកំណត់ Key Entities, Relationships ហើយ Database Schema និង Entity-Relationship (ER) Design ។ Advertisement System ប្រើប្រាស់ MongoDB, NoSQL database ដែលរក្សាទុកទិន្នន័យក្នុងទម្រង់តម្រង់ document-oriented ដែល Format Using Collections. Collection នីមួយៗតំណាងឱ្យ Entity នៅក្នុង System ដូចជា Advertisements និង LED Screens។ ហើយ Entities ទាំងនេះត្រូវបានទាក់ទងតាមរយៈ ObjectId ដើម្បី Maintain Data Integrity។

2.1. Entity Relationship (ER) Diagram

Entity Relationship Diagram (ER) គឺជា Conceptual Data Modeling Tool ដែល Represents ឱ្យ Structure នៃ System's Database។ វាបង្ហាញពី Entities ផ្សេងៗគ្នា (ដូចជា Users, Advertisements, LED Screens, និង Transactions) ទាក់ទងគ្នាទៅវិញទៅមក។ វាជួយ Developers និង Database Designers ឱ្យយល់អំពី Data Flow និង Relationships រវាងសមាសធាតុផ្សេងៗនៅក្នុងប្រព័ន្ធ។ ER Diagram ផ្តល់នូវ High-level View អំពីរបៀបដែលទិន្នន័យត្រូវបានរៀបចំ និងតភ្ជាប់នៅក្នុងប្រព័ន្ធ។ វា Serves ដូចទៅនឹង Blueprint សម្រាប់ Designing Database Schema ដោយធានាថាទិន្នន័យត្រូវបានរក្សាទុកប្រកបដោយប្រសិទ្ធភាព Relationships ត្រូវបានរក្សាទុកយ៉ាងត្រឹមត្រូវ និងប្រតិបត្តិការប្រព័ន្ធ (ដូចជាការគ្រប់គ្រងអ្នកប្រើប្រាស់ និងការបង្ហាញការផ្សាយពាណិជ្ជកម្ម) ដំណើរការយ៉ាងរលូន។

2.2. Entity: User

- Table Name: User
- Primary Key (PK): userId

Attributes

Field	Type	Constraints	Description
userId	ObjectId	Primary Key	Auto generated by MongoDB
name	String	required	User's full name
email	String	required, unique	Email address
password	String	required	Hashed password
isSSO	Boolean	default: false	Whether the user registered
profileUrl	String	optional	Profile picture URL
isVerified	Boolean	default: false	Email verification status
company	String	optional	Company name
contact	String	optional	Contact number
role	String	enum(admin, publisher)	User role
telegram.userId	String	default: null	Telegram user ID
telegram.isConnected	Boolean	default: false	Telegram connection status
telegram.chatId	String	default: null	Telegram chat ID
telegram.verifyToken	String	default: null	Token used for verifying Telegram link
createdAt, updatedAt	Date	auto	Timestamps for record.

2.3. Entity: Wallet

- Table Name: Wallet
- Primary Key (PK): walletId
- Foreign Key (FK): userId → User.userId

Attributes:

Field	Type	Constraints	Description
walletId	ObjectId	Primary Key	Unique wallet identifier (PK).
userId	ObjectId	Foreign Key	References the owner user (FK).
balance	Number	Default: 0	Current amount in the wallet.
createdAt, updatedAt	Date	auto	Timestamps for record.

2.4. Entity: Transaction

- Table Name: Transaction
- Primary Key (PK): transactionId
- Foreign Keys (FK):userId → User.userId, walletId → Wallet.walletId

Attributes:

Field	Type	Constraints	Description
transactionId	ObjectId	Primary Key	Unique transaction record ID
userId	ObjectId	Foreign Key	References the user who performed the transaction
walletId	ObjectId	Foreign Key	References the wallet used in the transaction
amount	Number	Required	Transaction amount
type	Boolean	Required	Type of transaction

orderId	String	optional	Reference order ID for linkage
productTitle	String	Optional	Related product or service title
description	String	optional	Additional details of the transaction
createdAt, updatedAt	Date	auto	Timestamps for record.

2.5. Entity: Ads

- Table Name: Ads
- Primary Key (PK): adsId
- Foreign Keys (FK): publisherId → User.userId, ledId → LedSchema.ledId

Attributes:

Field	Type	Constraints	Description
adsId	ObjectId	Primary Key	Unique advertisement identifier
publisherId	ObjectId	Foreign Key	References the publisher (User) who created the ad
ledId	ObjectId	Foreign Key	References the LED screen where the ad will display
title	String	Required	Ad title or campaign name
mediaUrl	String	Required	URL of image or video file
type	String	Required	Type of advertisement
duration	Number	Required	Duration of ad display (seconds)
displayTime	Date	Required	Scheduled display time
displayTime.startTime	Date	Required	Scheduled start time for display
displayTime.endTime	Date	Required	Scheduled end time for display
pricePerSecond	Number	Number	Cost rate per second

status	enum	Enum	Current ad state
billingStatus	enum	Enum	Billing/payment status
reviewStatus	enum	Enum	Review stage for approval
totalCost	Number	Calculated field	Total cost of the ad campaign
createdAt, updatedAt	Date	auto	Timestamps for record.

2.6. Entity: adsTransaction

- Table Name: adsTransaction
- Primary Key (PK): adsTransactionId
- Foreign Keys (FK): adsId → Ads.adsId, publisherId → User.userId

Attributes:

Field	Type	Constraints	Description
adsTransactionId	ObjectId	Primary Key	Unique advertisement transaction identifier
adsId	ObjectId	Foreign Key	References the related advertisement
publisherId	ObjectId	Foreign Key	References the publisher who owns the ad
adsTitle	String	Required	Title of the ad being displayed
pricePerSecond	Number	Required	Display price per second
totalCost	Number	Calculated , must be ≥ 0	Total cost (duration × pricePerSecond)
status	String	Requ default: <i>pending</i> ired	Display or payment status
displayTime	Date	Required	Scheduled date/time for the display
startTime	Date	Required	Actual start time of display

endTime	Date	Required	Actual end time of display
createdAt, updatedAt	Date	Auto	Record last update timestamp

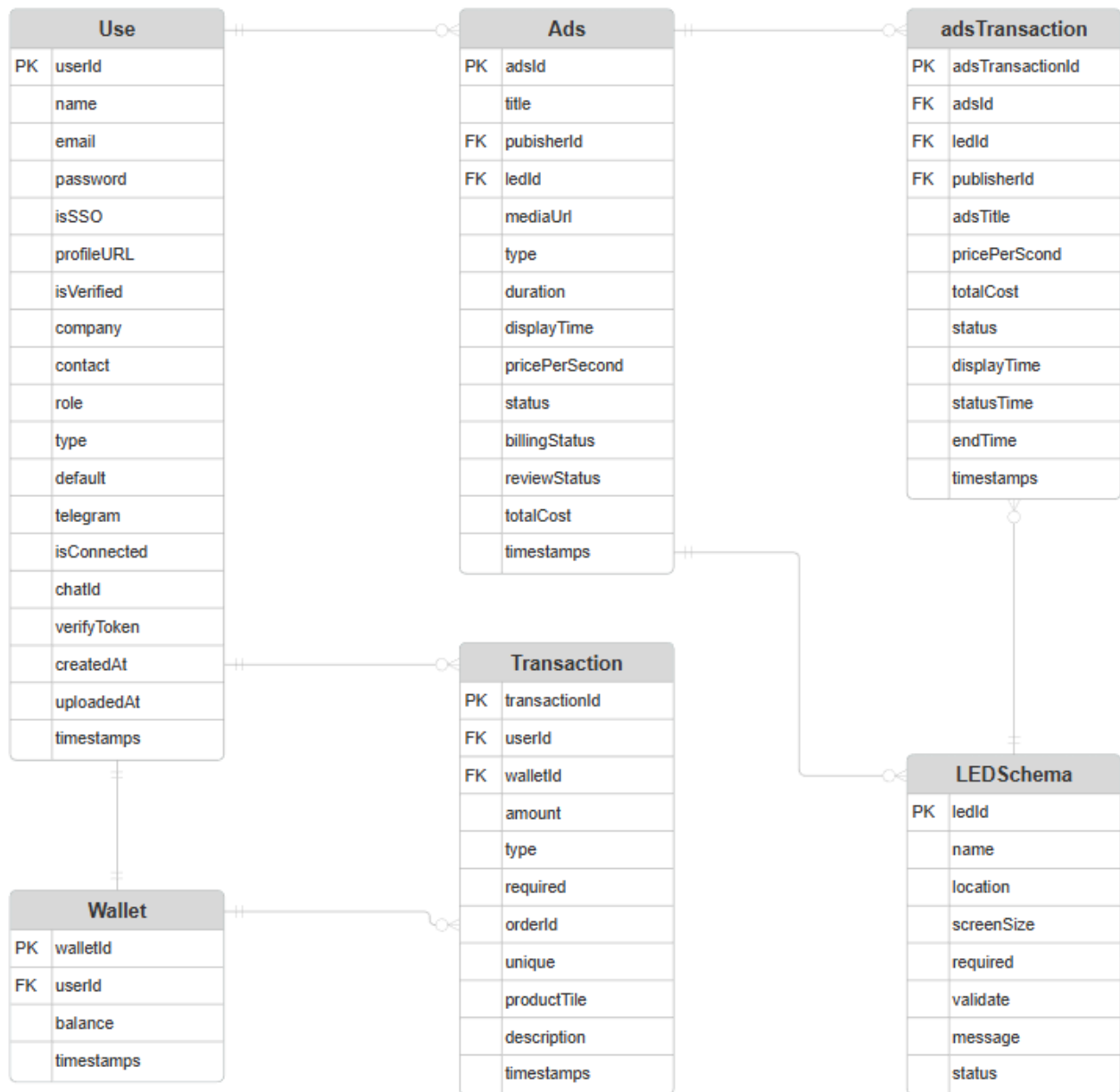
2.7. Entity: LedSchema

- Table Name: LedSchema
- Primary Key (PK): ledId

Attributes:

Field	Type	Constraints	Description
ledId	ObjectId	Primary Key	Unique identifier for each LED screen
name	String	Required , unique	Name of the LED
location	String	Required	Physical installation location
screenSize	String	Required	Physical size or resolution
status	String	default: <i>active</i>	Operational state of the LED
createdAt, updatedAt	Date	Auto	Record last update timestamp

3. ER diagrams, schema, and relationships



Entity-Relationship Diagram (ERD) នេះសង្ខេបជា system's core functionality នៃប្រព័ន្ធ គឺទាក់ទងនឹងការគ្រប់គ្រងអ្នកប្រើប្រាស់ និងសកម្មភាព Financial របស់ពួកគេតាមរយៈ Wallet និងតាម ដានការផ្សាយពាណិជ្ជកម្មពីការបង្កើតរហូតដល់បង្ហាញ។ ជាពិសេស អ្នកប្រើប្រាស់មាន Wallet តែមួយ

ហើយអាចចាប់ផ្តើមប្រតិបត្តិការច្រើន និង Publish ផ្សាយពាណិជ្ជកម្មផ្សេងៗ។ ការផ្សាយពាណិជ្ជកម្មទាំងនេះត្រូវបានកំណត់ពេលដំណើរការនៅលើ LedSchema ហើយព័ត៌មានលម្អិតនៃការប្រតិបត្តិរបស់ពួកគេ រួមទាំងការចំណាយ និងរយៈពេលត្រូវបានកត់ត្រានៅក្នុងតារាង AdsTransaction ដោយធានាថាកំណត់ត្រាពេញលេញនៃប្រវត្តិហិរញ្ញវត្ថុ និងប្រតិបត្តិការផ្សាយពាណិជ្ជកម្មត្រូវបានរក្សាទុក។

3.1. Relationship Summary

Relationship	Type	Description
User → Wallet	One to One	Each user has one wallet
User → Transaction	One to Many	A user can have multiple transactions
Wallet → Transaction	One to Many	A wallet can have multiple transactions
User → Ads	One to Many	A user (publisher) can create multiple ads
Ads → adsTransaction	One to Many	A user can have many ad transactions
adsTransaction → LedSchema	One to Many	Each ad can be part of many transactions
LedSchema → Ads	One to Many	Each LED screen can host multiple ads

4. UML Diagrams

Unified Modeling Language (UML) Diagrams គឺជាការបង្ហាញរូបភាពដែលប្រើដើម្បី Describe, Design, និងយល់ Structure និង Behavior នៃ System។ ជួយ Developers, Designers, និង Stakeholders មើលឃើញពីរបៀបដែល Components ផ្សេងគ្នានៃ System Interact ជាមួយគ្នា ដែលធ្វើឱ្យដំណើរការអភិវឌ្ឍន៍កាន់តែរៀបចំ និងមានប្រសិទ្ធភាព។

4.1. Class diagrams

Class diagram នេះកំណត់ឱ្យ Structure និង Relationships នៃ Main Components សំខាន់ៗ នៅក្នុង System។ វារួមបញ្ចូលទាំង 6 primary class User, Wallet, Transaction, Ads, LedSchema, និង AdsTransaction. Class នីមួយៗមាន Contains Attributes (Data Fields) និង Methods (Functions)។ Relationships បង្ហាញពីរបៀបដែល Class ទាំងនោះ Interact ជាមួយគ្នាទៅ វិញទៅមក។

4.2. User Class

Attributes

- userId : int (Primary)
- name : string
- email : string
- password : string
- isSSO : boolean
- profileUrl : string
- isVerified : boolean
- company : string
- contact : string
- role : string (admin, publisher)
- telegram : string/boolean
- createdAt, updatedAt : datetime

Methods

- User management: registerUser(), updateProfile(), changePassword()
- Security: sendVerificationEmail(), verifyToken(), resetPassword()
- Telegram link: setUserProfile(), updateUserProfile()

Relationship

- 1 User → 1 Wallet
- 1 User → Many Ads
- 1 User → Many Transactions

4.3. Wallet Class

Attributes

- walletId : int
- userId : int
- balance : float
- timestamps : datetime

Methods

- createWallet()
- deposit()
- deduct()

Relationship

- 1 User → 1 Wallet
- 1 Wallet → Many Transactions

4.4. Transaction Class

Attributes

- transactionId : int
- userId : int
- walletId : int
- type : string
- amount : float
- orderId : string
- productTitle : string
- description : string
- timestamps : datetime

Methods

- createTransaction()

Relationship

- 1 Wallet → Many Transactions
- 1 User → Many Transactions

4.5. Ads Class

Attributes

- adId : int
- title : string
- publisherId : int
- ledId : int
- mediaUrl : string
- type : string
- duration : int
- displayTime : int

- pricePerSecond : float
- status : string
- billingStatus : string
- reviewStatus : string
- totalCost : float
- timestamps : datetime

Methods

- Fetch ads: getAds(), getUserAds(), getAdminAds()
- Update: updateAds(), deleteLED(), updateLED(), approvalAdsViaBot()
- Analysis: getPendingAds(), reviewAds(), getBookedRanges()

Relationship

- 1 User (publisher) → Many Ads
- 1 LedSchema → Many Ads
- 1 Ads → Many AdsTransaction

4.6. LedSchema Class

Attributes

- ledId : int
- name : string
- location : string
- screenSize : string
- required : boolean
- validate : string
- message : string
- status : string

Relationship

- 1 LedSchema → Many Ads

4.7. AdsTransaction Class

Attributes

- adsTransactionId : int
- adId : int
- ledId : int
- publisherId : int
- pricePerSecond : float
- totalCost : float
- status : string
- displayTime : int
- startTime, endTime : datetime
- timestamps : datetime

Methods

- createAdsTransaction()
- getTransactionsByUser()

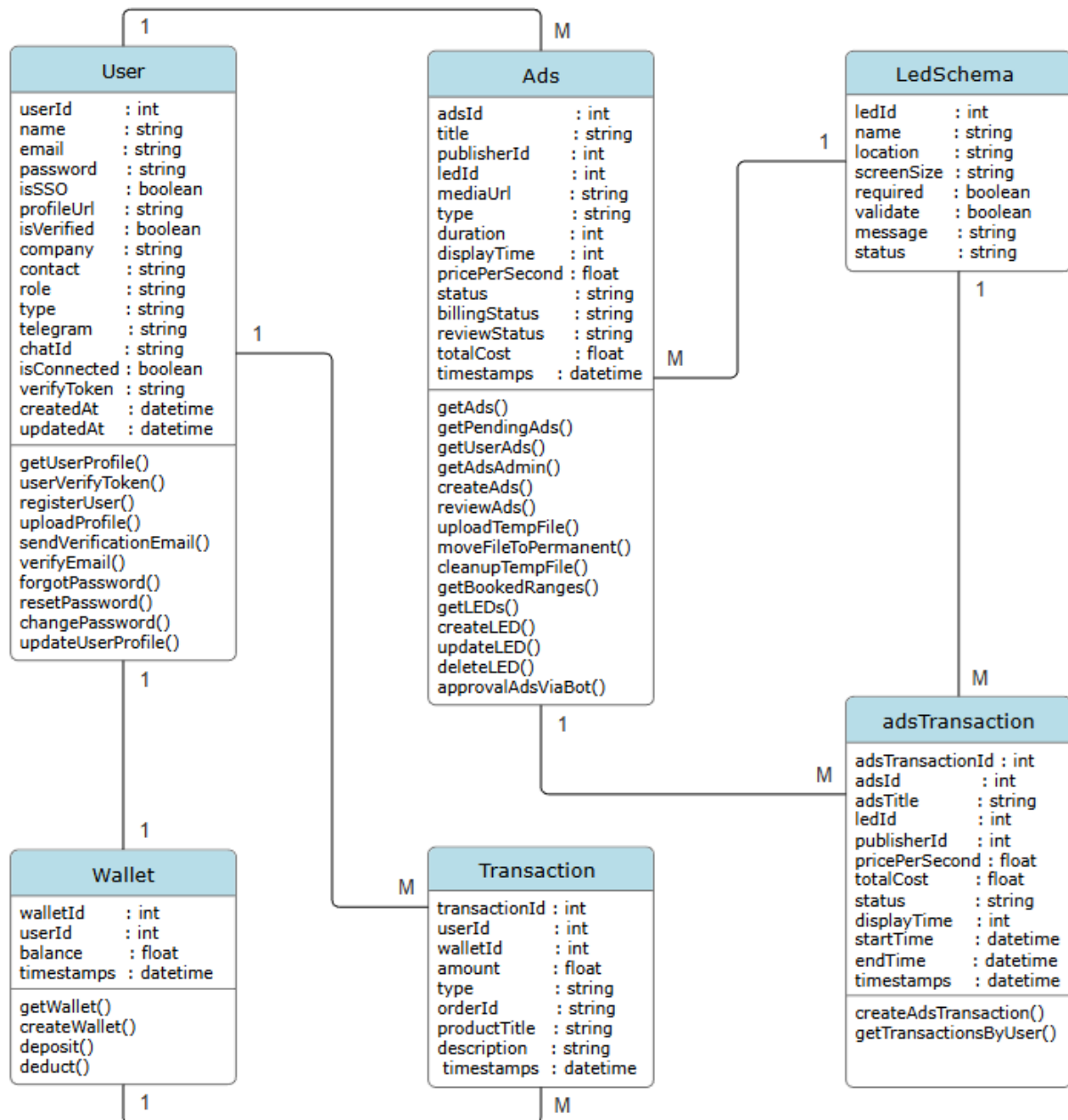
Relationship

- 1 Ads → Many AdsTransaction

❖ Overall Relationship Summary

- User → Wallet (1-to-1)
- User → Ads (1-to-Many)
- User → Transaction (1-to-Many)
- Wallet → Transaction (1-to-Many)
- LedSchema → Ads (1-to-Many)
- Ads → AdsTransaction (1-to-Many)

4.8. Class Diagram Structure



5. API Design Document

This section describes the design of all API endpoints exposed to the frontend through the API Gateway. It includes the HTTP method, endpoint URL, request headers, request body, response structure, and notes for developers. The table below summarizes each API endpoint and its expected behavior.

API Name	METHOD	API Gateway Endpoint	Request / Params	Response
Register	POST	/api/users/register	{ "name": "John Doe", "email": "john@example.com", "password": "Password@123" }	{ "success": true, "message": "User registered successfully", "id": "693cf447a7aa9422b177ecd7", "email": "chhievtong11@gmail.com", "role": "publisher", "wallet": "Wallet Created" }
Login	POST	/api/users/login	{ "email" : "chhievtong11@gmail.com", "password": "Tongtong000" }	{ "token": "eyJhbGciOiJIUzI1NiIsInR5cGE": "message": "Successfully logged in" }
Get User Profile	GET	/api/users/me		{ "success": true, "data": { "telegram": { "userId": null, "isConnected": false, "chatId": null, "verifyToken": null }, "_id": "693cf447a7aa9", "name": "john doe", "email": "chhievtong11@gmail.com", "isSSO": false, "profileUrl": "./default_avatar.jpg", "isVerified": false, "company": "", "contact": "", "role": "publisher", } }
Verify User Token	GET	/api/users/auth/verify		{ "valid": true, "user": { "id": "693cf447a7aa9422b177ecd7", "role": "publisher", "email": "chhievtong11@gmail.com" } }

Upload avatar image	POST	/api/users/upload	file : Image/video (JPEG/JPG/PNG,WEBP)	{ "message": "Successfully uploaded profile image", "userUpdatedProfile": { "telegram": { "userId": null, "isConnected": false, "chatId": null, "verifyToken": null }, "_id": "693cf447a7aa9422b177ecd7", "name": "john doe", "email": "chhievtong11@gmail.com", "isSSO": false, "profileUrl": "https://globaladvs- .png", "isVerified": false, "company": "", "contact": "", "role": "publisher", } }
Send verification Email	POST	/api/users /verify-email		{ "message": "Verification email sent" }
Send verify	POST	/api/users /verify/:token		{ "message": "Successfully verification" }
Forgot Password	POST	/api/users /forgot- password	{ "email": "chhievtong@gmail.com" }	{ "message": "Password reset link sent to your email." }
Change User Password	PUT	/api/users/change- password	{ "oldPassword": "Tongtong00", "newPassword": "Tongtong000" }	{ "message": "Incorrect old password" }
Get All Ads	GET	/api/ads/all		{ "message": "OK", "data": [{..}, {..}], "total": 0, "page": 1, "pages": 0 }
Get pending ads	GET	/api/ads/pending		{ "message": "OK", "data": [{..}, {..}], "total": 0, "page": 1, "pages": 0 }

Get user ads data	GET	/api/ads/me		{ "message": "OK", "data": [{..}, {..}], "total": 0, "page": 1, "pages": 0 }
Admin get all ads detail	GET	/api/ads/admin		{ "message": "OK", "data": [{ "displayTime": { "startTime": "2025-12-12T14:46:11.000Z", "endTime": "2025-12-12T14:47:13.000Z" }, "_id": "693c2a7dac0e3e1855d0bd6d", "title": "LED Advertisement - image (62s)", "publisherId": "68aadd882865541964e8bb0c", "led": "68b3277e4f40d66e80dc1a90", "mediaUrl": "https://... colormap.png", "type": "image", "duration": 62, "pricePerSecond": 0.5, "status": "pending", "billingStatus": "paid", "reviewStatus": "approved", "totalCost": 3 },] }
Create ads	POST	/api/ads/create/:led	{ "title": "testing", "publisherId": "693cf447a7aa9422b177ecd7", "mediaUrl": "image/url", "duration": 240, "displayTime": { "startTime": "2025-08-26T14:45:00.373Z", "endTime": "2025-08-26T14:50:00.373Z" }, "pricePerSecond": "1.02" }	{ "message": "Created", "data": { "title": "testing", "publisherId": "693cf447a7aa9422b177ecd7", "led": "68ac276f23e137f1bb2fbf42", "mediaUrl": "localhost:3000/image", "type": "image", "duration": 240, "displayTime": { "startTime": "2025-08-26T14:45:00.373Z", "endTime": "2025-08-26T14:50:00.373Z" }, "pricePerSecond": 1.02, "status": "draft", "billingStatus": "unpaid", "reviewStatus": "pending", "totalCost": 244.8, "_id": "693d1fee87cf000798700cdd", } }

Approval Ads	PUT	/api/ads/review/:adslid	{ "reviewStatus": "approved" "rejected" }	{ "message": "Ad reviewed successfully", "data": { "displayTime": { "startTime": "2025-08- 26T14:45:00.373Z", "endTime": "2025-08- 26T14:50:00.373Z" }, "_id": "693d1fee87cf000798700cdd", "title": "testing", "publisherId": "693cf447a7aa9422b177ecd7", "led": "68ac276f23e137f1bb2fbf42", "mediaUrl": "localhost:3000/image", "type": "image", "duration": 240, "pricePerSecond": 1.02, "status": "draft", "billingStatus": "unpaid", "reviewStatus": "approved", "totalCost": 244.8, } }
Upload image/video into temporary AWS folder	POST	/api/ads/upload-temp	file : Image/video (JPEG/JPG/PNG,WEBP),{MP4, MOV , AVI , QUICKTIME,X- MSVIDEO,WEBM)	{ "message": "Successfully uploaded temporary file", "tempKey": "tmp- uploads/68aadd8...-apple.jpg", "fileUrl": "https://...png ", "metadata": { "userId": "68aadd882865541964e8bb0c", "originalName": "apple.jpg", "timestamp": 1765614656259, "fileType": "image/jpeg", "fileSize": 64693 } }
Get Specific LED Time	GET	/api/ads/book- rangs?led=.....		{ "bookedRanges": [{.}, {...}] }
Get All LED	GET	/api/leds/all		[{ "_id": "68ac276f23e137f1bb2fbf42", "name": "LED00", "location": "Phnom Penh", "screenSize": "1280x960", "status": "active", },]
Create LED Display	POST	/api/leds/create	{ "name" : "LED04", "location" : "Time Squire, NY ", "screenSize" : "1280x780" }	{ "name" : "LED04", "location" : "Time Squire, NY ", "screenSize" : "1280x780", "status": "active", "pricing": "6922bf9dbc4094bbd5a7dc8c", "_id": "6922bf9dbc4094bbd5a7dc92", }

Update LED	PUT	/api/leds/update/:id		{ "message" : "successfully updated", "led" : {...} }
Delete LED	DELETE	/api/leds/delete/:id		{ "message" : "successfully deleted !", "led" : {...} }